Problem Statement & Approach

As a Data Analyst at a financial institution, the goal is to predict whether a loan application will be approved (1) or not approved (0) from customer details. The workflow below covers data loading, cleaning, EDA, outlier treatment, encoding, scaling, Logistic Regression modelling, and evaluation (Confusion Matrix, ROC and AUC).

How to use this document: paste each code block into consecutive cells of a Google Colab notebook and run them in order. The code auto-detects the target column and the Years of Employment column, so it works even if your CSV uses slightly different names. After running, copy your actual outputs into the "Answer / Interpretation" boxes (guidance on what to write is provided).

Submission checklist

- Google Colab notebook (code cells in the order given here)
- Exploratory Data Analysis (Q3 plots)
- Model evaluation metrics (Q9 and Q10)
- Regression equation (Section 11)
- Interpretation of coefficients (Section 11)

Setup: Libraries and Dataset

```
import pandas as pd, numpy as np
import matplotlib.pyplot as plt, seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (confusion_matrix, ConfusionMatrixDisplay, accuracy_score,
                             precision_score, recall_score, f1_score, roc_curve,
                             roc_auc_score, classification_report)
import warnings; warnings.filterwarnings('ignore')
sns.set_style('whitegrid')

# --- Upload the CSV downloaded from the dataset link (Colab) ---
from google.colab import files
uploaded = files.upload()                # choose the loan approval CSV
file_name = list(uploaded.keys())[0]
```

Q1. Data Loading & Understanding

a) Load the dataset

```
df = pd.read_csv(file_name)
df.columns = df.columns.str.strip()          # remove stray spaces in column names
```

b) Display the first 5 rows

```
df.head()
```

c) Shape of the dataset

```
print("Shape (rows, columns):", df.shape)
```

d) Data types of each column

```
print(df.dtypes)
df.info()
```

Answer / Interpretation: State the number of rows and columns from df.shape, and note which columns are numeric (int64/float64) and which are object (text). The target column (e.g., loan_status / Loan_Status) tells whether the loan was Approved or Rejected.

Q2. Data Cleaning

a) Check for missing values

```
print(df.isnull().sum())
print("\nDuplicate rows:", df.duplicated().sum())
```

b) Handle missing values

Numeric columns are filled with the median (robust to outliers); categorical columns are filled with the mode (most frequent value).

```
# Identify target column automatically
target_candidates = [c for c in df.columns if 'status' in c.lower() or 'approv' in c.lower()]
target = target_candidates[0]
print("Target column:", target)

num_cols = df.select_dtypes(include=np.number).columns.tolist()
cat_cols = df.select_dtypes(include='object').columns.tolist()

for c in num_cols:
    if df[c].isnull().sum() > 0:
        df[c] = df[c].fillna(df[c].median())
for c in cat_cols:
```

```
    if df[c].isnull().sum() > 0:
        df[c] = df[c].fillna(df[c].mode()[0])
    df[c] = df[c].astype(str).str.strip()     # clean stray spaces in text values

df.drop_duplicates(inplace=True)
print("Missing values after cleaning:", df.isnull().sum().sum())
```

c) Identify categorical columns

```
cat_cols = df.select_dtypes(include='object').columns.tolist()
print("Categorical columns:", cat_cols)
```

Answer / Interpretation: Mention which columns had missing values, the method used (median for numeric, mode for categorical) and why. List the categorical columns printed by the code (typically education, self-employed and the target loan status).

Q3. Exploratory Data Analysis (EDA)

a) Distribution of the target variable

```
plt.figure(figsize=(6,4))
sns.countplot(x=target, data=df, palette='Set2')
plt.title('Distribution of Loan Status'); plt.show()
print(df[target].value_counts(normalize=True)*100)
```

Answer / Interpretation: Report the percentage of Approved vs Rejected loans. If one class is clearly larger (e.g., ~60/40), the data is moderately imbalanced, so accuracy alone is not enough and Precision, Recall, F1 and AUC should also be used.

b) One numerical feature vs target

A strong, commonly used choice is the applicant income or credit/CIBIL score; the code picks the first suitable column automatically.

```
pref = ['cibil','credit','income','loan_amount']
num_cols = df.select_dtypes(include=np.number).columns.drop(target,
errors='ignore').tolist()
feat = next((c for k in pref for c in num_cols if k in c.lower()), num_cols[0])
print("Numerical feature analysed:", feat)

fig, ax = plt.subplots(1, 2, figsize=(12,4))
sns.boxplot(x=target, y=feat, data=df, ax=ax[0], palette='Set2')
ax[0].set_title(f'{feat} vs {target} (Boxplot)')
sns.histplot(data=df, x=feat, hue=target, kde=True, ax=ax[1])
```

```
ax[1].set_title(f'Distribution of {feat} by {target}')
plt.tight_layout(); plt.show()
print(df.groupby(target)[feat].describe())
```

Answer / Interpretation: Compare the median/mean of the chosen feature for approved vs rejected applicants. Typically approved applicants have a higher credit score / income, indicating the feature is a useful predictor.

c) Years of Employment vs target

```
emp_candidates = [c for c in df.columns if 'employ' in c.lower() and c in num_cols]
emp = emp_candidates[0] if emp_candidates else None
print("Employment column:", emp)

if emp:
    fig, ax = plt.subplots(1, 2, figsize=(12,4))
    sns.boxplot(x=target, y=emp, data=df, ax=ax[0], palette='Set2')
    ax[0].set_title(f'{emp} vs {target}')
    sns.barplot(x=target, y=emp, data=df, ax=ax[1], palette='Set2')
    ax[1].set_title(f'Average {emp} by {target}')
    plt.tight_layout(); plt.show()
    print(df.groupby(target)[emp].mean())
else:
    print("No numeric employment column found - check df.columns and set emp
manually.")
```

Answer / Interpretation: State whether applicants with more years of employment are approved more often (compare the average years for each group). Longer, stable employment generally signals lower default risk, hence a higher approval chance.

Additional EDA: correlation heatmap

```
tmp = df.copy()
tmp[target] = tmp[target].astype('category').cat.codes
plt.figure(figsize=(10,7))
sns.heatmap(tmp.select_dtypes(include=np.number).corr(), annot=True, fmt='.2f',
cmap='coolwarm')
plt.title('Correlation Heatmap'); plt.show()
```

Q4. Outlier Detection and Treatment

a) Detect outliers using the IQR method

```
num_cols = df.select_dtypes(include=np.number).columns.drop(target,
errors='ignore').tolist()
bounds = {}
for c in num_cols:
    Q1, Q3 = df[c].quantile(0.25), df[c].quantile(0.75)
    IQR = Q3 - Q1
    lo, hi = Q1 - 1.5*IQR, Q3 + 1.5*IQR
    bounds[c] = (lo, hi)
    n_out = ((df[c] < lo) | (df[c] > hi)).sum()
    print(f"{c:30s} outliers: {n_out:5d}    (lower={lo:.2f}, upper={hi:.2f})")

df[num_cols].plot(kind='box', subplots=True, layout=(3, int(np.ceil(len(num_cols)/3))),
                  figsize=(15,9), title='Boxplots before capping')
plt.tight_layout(); plt.show()
```

b) Treat outliers using capping (winsorization)

```
for c, (lo, hi) in bounds.items():
    df[c] = df[c].clip(lower=lo, upper=hi)

remaining = {c: int(((df[c] < lo) | (df[c] > hi)).sum()) for c,(lo,hi) in bounds.items()}
print("Outliers remaining after capping:", remaining)
```

Answer / Interpretation: List the columns that had outliers and how many. Values below the lower fence are set to the lower fence and values above the upper fence to the upper fence (Q1 - 1.5*IQR, Q3 + 1.5*IQR). Capping keeps all rows, unlike deletion, so no data is lost.

Q5. Convert Target to Numeric and Drop Unnecessary Columns

```
# Drop ID-like columns (no predictive value)
id_cols = [c for c in df.columns if 'id' == c.lower() or c.lower().endswith('_id') or c.lower()
in ('id','loan_id','customer_id')]
df.drop(columns=id_cols, inplace=True)
print("Dropped columns:", id_cols)

# Encode target: Approved -> 1, Rejected / Not Approved -> 0
def encode_target(v):
    v = str(v).strip().lower()
    return 1 if v in ('approved','y','yes','1','true') else 0
df[target] = df[target].apply(encode_target)
print(df[target].value_counts())

# Encode remaining categorical predictors (one-hot, drop_first avoids dummy trap)
```

```
cat_cols = df.select_dtypes(include='object').columns.tolist()
df = pd.get_dummies(df, columns=cat_cols, drop_first=True, dtype=int)
df.head()
```

Answer / Interpretation: The target is now numeric (Approved = 1, Not Approved = 0). Identifier columns such as Loan_ID were dropped because they do not carry predictive information. Remaining categorical features were one-hot encoded so the model can use them.

Q6. Feature Selection and Data Splitting

a) Separate X and y

```
X = df.drop(columns=[target])
y = df[target]
print("X shape:", X.shape, "| y shape:", y.shape)
```

b) Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print("Train:", X_train.shape, " Test:", X_test.shape)
```

Answer / Interpretation: 80% of the data is used for training and 20% for testing. stratify=y keeps the approved/rejected ratio the same in both sets, and random_state=42 makes results reproducible.

Q7. Feature Scaling using StandardScaler

```
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)   # fit ONLY on training data
X_test_sc  = scaler.transform(X_test)        # apply same transformation to test data
print("Train mean ~", X_train_sc.mean().round(3), "| Train std ~",
X_train_sc.std().round(3))
```

Answer / Interpretation: StandardScaler transforms each feature to mean 0 and standard deviation 1: z = (x - mean) / std. The scaler is fitted only on the training set to avoid data leakage. Scaling helps logistic regression converge and makes coefficients comparable across features.

Q8. Logistic Regression Model Building

a) Train the model

```
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train_sc, y_train)
print("Training accuracy:", model.score(X_train_sc, y_train).round(4))
```

b) Predict on the test data

```
y_pred = model.predict(X_test_sc)
y_prob = model.predict_proba(X_test_sc)[:, 1]     # probability of approval
print("Test accuracy:", accuracy_score(y_test, y_pred).round(4))
```

Q9. Model Evaluation: Confusion Matrix

a) Generate the confusion matrix

```
cm = confusion_matrix(y_test, y_pred)
print(cm)
ConfusionMatrixDisplay(cm, display_labels=['Not Approved (0)','Approved
(1)']).plot(cmap='Blues')
plt.title('Confusion Matrix'); plt.show()

TN, FP, FN, TP = cm.ravel()
print(f"TN={TN}  FP={FP}  FN={FN}  TP={TP}")
print("Accuracy :", round(accuracy_score(y_test, y_pred),4))
print("Precision:", round(precision_score(y_test, y_pred),4))
print("Recall   :", round(recall_score(y_test, y_pred),4))
print("F1-score :", round(f1_score(y_test, y_pred),4))
print("\n", classification_report(y_test, y_pred))
```

b) Interpret the results

- True Negatives (TN): rejected loans correctly predicted as rejected.
- True Positives (TP): approved loans correctly predicted as approved.
- False Positives (FP): loans predicted approved but actually rejected. This is risky for the bank (possible bad loans / defaults).
- False Negatives (FN): loans predicted rejected but actually approved. This means lost business and unfairly rejected good customers.
- Accuracy = (TP+TN)/Total; Precision = TP/(TP+FP); Recall = TP/(TP+FN); F1 = harmonic mean of Precision and Recall.

Answer / Interpretation: Insert your actual TN, FP, FN, TP values and the metrics printed above, then conclude, for example: "The model achieves X% accuracy; because FP errors are

costlier for a lender, precision should be monitored closely, and the decision threshold can be raised above 0.5 to reduce FP."

Q10. Model Evaluation: ROC Curve & AUC

a) Plot the ROC curve

```
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
auc = roc_auc_score(y_test, y_prob)

plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'Logistic Regression (AUC = {auc:.3f})')
plt.plot([0,1],[0,1],'k--', label='Random guess (AUC = 0.5)')
plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate (Recall)')
plt.title('ROC Curve'); plt.legend(loc='lower right'); plt.show()
```

b) Calculate the AUC score

```
print("AUC Score:", round(auc, 4))
```

Answer / Interpretation: AUC measures how well the model separates approved from rejected applicants across all thresholds: 0.5 = random guessing, 0.7-0.8 = acceptable, 0.8-0.9 = good, above 0.9 = excellent. A curve bending toward the top-left corner indicates a strong classifier. Write your AUC value and the matching quality level.

11. Regression Equation & Interpretation of Coefficients

Regression equation

Logistic regression models the log-odds of approval: ln(p / (1 - p)) = b0 + b1*x1 + b2*x2 + ... + bk*xk, where p = P(Loan approved = 1). Equivalently p = 1 / (1 + e^-(b0 + b1x1 + ... + bkxk)).

```
coef_df = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_[0],
    'Odds_Ratio (exp(coef))': np.exp(model.coef_[0])
}).sort_values('Coefficient', ascending=False)
print("Intercept (b0):", round(model.intercept_[0], 4))
print(coef_df.to_string(index=False))

# Print the regression equation
terms = " ".join([f"{c:+.3f}*[{f}]" for f, c in zip(X.columns, model.coef_[0])])
print("\nln(p/(1-p)) = {:.3f} {}".format(model.intercept_[0], terms))
```

```
coef_df.set_index('Feature')['Coefficient'].sort_values().plot(kind='barh', figsize=(8,6),
color='steelblue')
plt.title('Logistic Regression Coefficients (standardised features)'); plt.show()
```

How to interpret the coefficients

- Sign: a positive coefficient increases the log-odds (and hence probability) of loan approval; a negative coefficient decreases it.
- Magnitude: because features are standardised, a larger absolute coefficient means a stronger influence on the decision.
- Odds ratio = exp(coefficient): a value above 1 means that a 1-standard-deviation increase in the feature multiplies the odds of approval by that factor; below 1 reduces the odds. Example: an odds ratio of 1.5 means the odds of approval rise by 50%.
- Typical pattern in loan data: credit/CIBIL score and income have strongly positive coefficients, whereas a high loan amount or longer loan term tends to be negative. Higher years of employment generally has a positive effect.

Answer / Interpretation: Write the top 3 positive and top 3 negative features from your coef_df table with their odds ratios, and explain in plain business language what each implies for loan approval.

Conclusion

A Logistic Regression model was built after cleaning the data, treating outliers with IQR capping, encoding categorical variables, splitting the data 80/20, and scaling features with StandardScaler. Model quality was assessed through the confusion matrix (accuracy, precision, recall, F1) and the ROC curve with its AUC score. The coefficients show which customer attributes drive loan approval, so the bank can use the model to support faster, more consistent lending decisions. Possible improvements include trying other thresholds, class weights for imbalance, and comparing with tree-based models.